



מהנדסים טאלנטיים בתוכנה

מבוא לתכנות מערכות תרגיל בית מספר 3

מועד פרסום: 1.6.2026 בשעה 12:00

מועד הגשה: 18.6.2026 בשעה 23:59

1 הערות כלליות

- שימו לב! הפעם לא יינתנו דחיות במועד הגשת התרגיל. תכננו את זמנכם בהתאם
- בכל שאלה בנוגע לתרגיל הבית מומלץ ורצוי לפנות בקבוצת הדיסקורד. בפניות באמצעות דוא"ל יש לכתוב בשורת הנושא (Subject): תכנות 2 תרגיל 3
- מומלץ לקרוא ולהבין את כל ההנחיות לתרגיל לפני תחילת הפתרון. שאלות אשר התשובה עליהן מופיעה בגיליון התרגיל לא תיענינה!
- הגשת התרגיל תתבצע בזוגות.
- התרגיל מנוסח בלשון זכר אך מכוון לכל המינים.

2 נושא התרגיל – פיתוח מערכת לניהול חברות תחבורה ציבורית

3 מטרת התרגיל

תכנון, כתיבה ומימוש של ADT. עבודה עם ממשקי ADT אשר מימושם לא נתון. תכנון וכתיבת ADT המשתמשים ב – ADTs אחרים (Nested ADTs). שימוש ב – ADTs גנריים. שימוש במצביעים לפונקציות ועבודה עם Makefile

4 חלק א (10%)

4.1 סעיף א

ראינו שקיימים טיפוסים נתונים מופשטים ADTs שונים המאפשרים אחסון של נתונים מסוגים שונים. נתונים ה ADTs הבאים:

1. **מחסנית** – מאפשרת הכנסת נתונים והוצאתם בסדר הפוך לזה שהוכנסו. LIFO – האחרון שנכנס הוא הראשון לצאת
2. **רשימה מקושרת** – מאפשרת שמירה ממוינת של נתונים
3. **קבוצה** – שמירה של נתונים ללא חשיבות לסדר וללא כפילויות

הנכם נדרשים להציע מבנה נתונים מתאים עבור כל אחת מהבעיות המופיעות מטה. אנא נמקו תשובתכם בקצרה

א. חברת השליחויות סי-יו מעוניינת בפלטפורמה שמאפשרת להוסיף ולהסיר נקודות איסוף וחלוקה בזמן אמת. על המערכת לשמור את נקודות האיסוף ולהציג אותן לשליח על פי סדר שנקבע מראש

```
typedef struct centrifuge_s *Centr;
#define NUM_NUMBERS 9

// ...
// Check configuration by getting the numbers
CentrResult centrifuge_get_numbers(Centr c, /*out*/ unsigned int *config_numbers[]);
// ...
```

file: **centr.c**

```
// ...
#include "centr.h"

struct loto_s {
    unsigned int numbers[NUM_NUMBERS];
    // ...
};

CentrResult centrifuge_get_numbers(Centr c, unsigned int *config_numbers[]) {
// ...
    if (config_numbers != NULL) {
        *config_numbers = c->numbers;
        Return CENTR_SUCCESS;
    }
// ...
}

// Amit Campel /\
```

5 חלק ב (90%)

5.1 תיאור התרגיל

לאור ההצלחה הרבה במימוש המערכת לניהול התחבורה הציבורית שפיתחתם, מדינת סומלילנד חתמה על שדרוג מערכות התחבורה הציבורית שלה.

בנוסף, ממשלת סומלילנד מעוניינת במערכת חדשה ומשופרת לניהול כל חברות התחבורה במדינה. מניסיון העבר של הממשלה, ורצונה להשתמש במודולים מסוימים במקומות אחרים, היא מעוניינת בשיפורים הבאים באיכות הקוד:

- קריאות הקוד – על מנת להקל על ההבנה של התוכנית שלכם, הקפידו על מתן שמות משמעותיים למשתנים ופונקציות, המנעו מכתובת פונקציות ארוכות, המנעו משימוש בקבועים ללא שמות והמנעו משימוש במשתנים גלובליים
- מודולריות – הקפידו על חלוקת הקוד למודולים במבנה הגיוני, כך שפונקציות ומבני נתונים קשורים יהיו באותו מודול.
- שימוש חוזר בקוד – בצעו שימוש חוזר בקוד ככל הניתן והמנעו משכפול קוד. מסופקים לכם עם תרגיל הבית שני מודולים. **עליכם להשתמש באחד מהם לפחות** במהלך הפיתוח. המודולים מממשים את ה-ADTs הבאים: רשימה מקושרת (**LinkedList**) וקבוצה (**Set**). המודולים כוללים קבצי כותרת (**.h**) וקובץ ספרייה **libprog2.a** בלבד. ללא קבצי "**.c**". כלומר – המימוש של המודולים הללו אינו נתון לכם.
- עבור ה-**LinkedList ADT**, קיימת דוגמה לשימוש בתיקייה הנקראת **people_example**. ניתן להתרשם כיצד ניתן להשתמש בצורה נכונה ב-**ADT** הזה. שימוש ב-**Set ADT** מתבצע באופן דומה.
- שימו לב! **אין לממש רשימות מקושרות בתרגיל זה**. עליכם להשתמש במודולים המסופקים בלבד.

5.2 תיאור המערכת

- עליכם לכתוב מודול חדש שיממש את הממשק המפורט מטה. גם הפעם לא תספקו את פונקציית main, פונקציה זו תתווסף בזמן הבדיקה על ידי הבודקים. עם זאת, לשם בדיקה עצמית של התוכנית שלכם, תוכלו לכתוב פונקציית main משלכם בקובץ נפרד ולא להגיש אותה.
- **במימוש הפקודות אין הגבלות על כמות חברות התחבורה או על האובייקטים האחרים אלא אם צויין אחרת.**
- מכיוון שהבדיקה תתבצע באמצעות קוד, באופן אוטומטי, אין לנתח או לקבל קלט מהמשתמש.
- על המערכת להיות ממומשת בתצורת ADT ולממש את כל הפונקציות המופיעות בממשק transport_manager.h המסופק לכם. תיאור הפונקציות מופיע בסעיף 5.4 "פקודות"

5.3 טיפול בשגיאות

- גם בתרגיל זה אינכם מדפיסים הודעות שגיאה אלא נדרשים להחזיר את השגיאה המתאימה בערך החזרה של הפונקציות במקרה והתרחשה.
- אם מופיעות כמה שגיאות יש לדווח על השגיאה החשובה ביותר. סדר החשיבות מוגדר בהגדרת הטיפוס TransportResult הנמצא בקובץ prog2_ex3.h.
- במידה ומתגלה שגיאה בפקודה המבוקשת, אין לבצע אותה אלא רק להחזיר את הערך המתאים

השגיאות הבאות אינן מפורטות מטה בכל פקודה אך רלוונטיות לכל הפקודות:

- במקרה שאחד או יותר מהפרמטרים המתקבלים הוא NULL, יש להחזיר את השגיאה TRANSPORT_NULL_ARGUMENT.
- במקרה של כשלון בהקצאת זיכרון, יש להחזיר את השגיאה TRANSPORT_OUT_OF_MEMORY, אין לצאת מהתוכנית.
- במקרה של הצלחה, יש להחזיר TRANSPORT_SUCCESS

5.4 פקודות

על המערכת לממש את הפונקציות הבאות (מוגדרות גם ב transport_manager.h).
טיפוסי המשתנים מופיעים בקובץ prog2_ex3.h אלא אם כתוב אחרת.

5.5.1 יצירת מערכת ניהול חברות תחבורה חדשה

```
TransportManager TransportManagerCreate();
```

תיאור הפעולה: פעולה זו יוצרת מערכת לניהול חברות תחבורה.

פרמטרים:

- אין

ערכי החזרה:

- TransportManager מצביע למבנה נתונים חדש ומאותחל במקרה של הצלחה
- NULL במקרה של כשלון

5.5.2 שחרור מערכת לניהול חברות תחבורה

```
void TransportManagerDestroy(TransportManager tm);
```

תיאור הפעולה: שחרור כל משאבי המערכת שהוקצו בזמן הריצה ולא שוחררו עדיין.

פרמטרים:

- tm מצביע למבנה אותו יש לשחרר

ערכי החזרה:

- אין

5.5.3 הוספת חברת תחבורה למערכת

```
TransportResult TransportManagerAdd(TransportManager tm,
    const char *name, int id, TransportType type, double revenue,
    double expenses);
```

תיאור הפעולה: פעולה זו מוסיפה חברת תחבורה למאגר הנתונים. שימו לב שאין הגבלה על כמות החברות שניתן לקלוט למאגר

פרמטרים:

- tm - מצביע למבנה הנתונים.

- name - שם החברה שנרצה להוסיף.

- id - מספר מזהה של חברת התחבורה - ייחודי במערכת. מספר גדול ממש מ-0.

- type - סוג החברה החדשה. אחר מהבאים: FLIGHT, LIGHT_TRAIN, BUS, TRAIN, SHIPPING, TAXI.

הגדרת הטיפוס TransportType מופיעה בקובץ transport_manager.h

- revenue - סך הכנסות החברה בשנה. מספר עשרוני גדול או שווה ל-0

- expenses - סך הוצאות החברה בשנה. מספר עשרוני גדול או שווה ל-0

ערכי החזרה:

- TRANSPORT_INVALID_ID - המספר המזהה אינו תקין

- TRANSPORT_INVALID_TYPE - טיפוס החברה אינו תקין

- TRANSPORT_INVALID_REVENUE - סך ההכנסות השנתיות אינו תקין

- TRANSPORT_INVALID_EXPENSES - סך ההוצאות השנתיות אינו תקין

- TRANSPORT_ALREADY_EXISTS - חברה בעלת מספר מזהה זהה כבר קיימת במערכת

5.5.4 הסרת חברת תחבורה מהמערכת

```
TransportResult TransportManagerRemove(TransportManager tm, int id);
```

תיאור הפעולה: פעולה זו מסירה חברת תחבורה מהמערכת.

פרמטרים:

- tm - מצביע למבנה הנתונים.

- id - מספר מזהה של חברת התחבורה - ייחודי במערכת. מספר גדול ממש מ-0.

ערכי החזרה:

- TRANSPORT_INVALID_ID - המספר המזהה אינו תקין

- TRANSPORT_DOESNT_EXIST - חברה בעלת מספר מזהה זהה אינה קיימת במערכת

5.5.5 עדכון הכנסות שנתי עבור חברת תחבורה

```
TransportResult TransportManagerUpdateRevenue(TransportManager tm, int id,
double revenue);
```

תיאור הפעולה: עדכון הכנסות שנתיות לחברת תחבורה מסויימת.

פרמטרים:

- tm - מצביע למבנה הנתונים.
- id - מספר מזהה של חברת התחבורה - ייחודי במערכת. מספר גדול ממש מ-0.
- revenue - סך הכנסות החברה בשנה. מספר עשרוני גדול או שווה ל-0

ערכי החזרה:

- [TRANSPORT_INVALID_ID](#) - המספר המזהה אינו תקין
- [TRANSPORT_INVALID_REVENUE](#) - סך ההכנסות השנתיות אינו תקין
- [TRANSPORT_DOESNT_EXIST](#) - חברה בעלת מספר מזהה זהה אינה קיימת במערכת

5.5.6 עדכון הוצאות שנתי עבור חברת תחבורה

```
TransportResult TransportManagerUpdateExpenses(TransportManager tm, int id,
double expenses);
```

תיאור הפעולה: עדכון הכנסות שנתיות לחברת תחבורה מסויימת.

פרמטרים:

- tm - מצביע למבנה הנתונים.
- id - מספר מזהה של חברת התחבורה - ייחודי במערכת. מספר גדול ממש מ-0.
- expenses - סך הוצאות החברה בשנה. מספר עשרוני גדול או שווה ל-0

ערכי החזרה:

- [TRANSPORT_INVALID_ID](#) - המספר המזהה אינו תקין
- [TRANSPORT_INVALID_EXPENSES](#) - סך ההכנסות השנתיות אינו תקין
- [TRANSPORT_DOESNT_EXIST](#) - חברה בעלת מספר מזהה זהה אינה קיימת במערכת

5.5.7 מיזוג מערכות לניהול חברות תחבורה

```
TransportResult TransportManagerMerge(TransportManager tm1, TransportManager tm2,
TransportManager *merged);
```

תיאור הפעולה: מיזוג שתי מערכות ניהול למערכת אחת המכילה את כל חברות התחבורה משתי המערכות. הפעולה תיכשל במידה וקיימות חברות תחבורה בעלות מספר מזהה זהה בשתי המערכות.

- במידה והפעולה לא מצליחה, שתי המערכות נשארות כפי שהיו ולא נוצרת אחת חדשה
- במידה והפעולה כן מצליחה, נוצרת מערכת ניהול חדשה המכילה את כל החברות משתי המערכות. בפרט, המשתמש יצטרך לקרוא ל `TransportManagerDestroy` גם עבור המערכת החדשה. כל המשאבים שהוקצו לשתי המערכות המקוריות משוחררים והמערכות כבר לא ניתנות לשימוש

פרמטרים:

- tm1 - מצביע למבנה הנתונים של המערכת הראשונה.
- tm2 - מצביע למבנה הנתונים של המערכת השנייה.
- merged - (פרמטר חזרה) המערכת החדשה שנוצרה עם איחוד שתי המערכות

ערכי החזרה:

- [TRANSPORT_ALREADY_EXISTS](#) - קיימת חברת תחבורה בעלת מספר מזהה זהה בשתי המערכות

5.5.8 דווח פרטי חברות התחבורה מסוג מסויים

```
TransportResult TransportManagerReportTransportCompanies(TransportManager tm,
    TransportType type, FILE *outChannel);
```

תיאור הפעולה: הדפסת פרטי חברות התעופה מהסוג המבוקש. הדפסת פרטי כל חברה תתבצע באמצעות הפונקציה `prog2_report_company()` המוגדרת בקובץ `prog2_ex3.h`. החברות יודפסו בסדר עולה על פי המספר המזהה שלהן.

פרמטרים:

- tm - מצביע למבנה הנתונים.
- type - סוג החברה החדשה. אחר מהבאים: FLIGHT, LIGHT_TRAIN, BUS, TRAIN, SHIPPING, TAXI. הגדרת הטיפוס `TransportType` מופיעה בקובץ `transport_manager.h`.
- outChannel - ערוץ הפלט אליו יש לשלוח את הדווח. במקרה כזה יודפסו כלל חברות התחבורה במאגר `TRANSPORT_ALL`, להיות גם `TRANSPORT_EMPTY`.

ערכי החזרה:

- `TRANSPORT_INVALID_TYPE` – טיפוס החברה אינו תקין
- `TRANSPORT_EMPTY` – אין חברות מהטיפוס המבוקש או בכלל.

5.5.9 דווח על כל חברות התחבורה הלא רווחיים במערכת

```
TransportResult TransportManagerReportUnprofitableCompanies(TransportManager tm,
    FILE *outChannel);
```

תיאור הפעולה: פקודה זו מדפיסה לערוץ הפלט את פרטי כל חברות התחבורה שההכנסה השנתית שלהם קטנה או שווה להוצאה השנתית. החברות יודפסו בסדר עולה על פי המספר המזהה שלהן. ההדפסה של כל חברה תתבצע באמצעות הפונקציה `prog2_report_company()` המוגדרת בקובץ `prog2_ex3.h`.

פרמטרים:

- tm - מצביע למבנה הנתונים
- outChannel - ערוץ הפלט אליו יש לשלוח את הדווח

ערכי החזרה:

- `TRANSPORT_EMPTY` – אין חברות מהטיפוס המבוקש או בכלל.

5.5.10 דווח על כל הסניפים מטיפוס מסוים במדינה

```
TransportResult TransportManagerReportCompaniesByNetIncome (TransportManager tm,
    FILE *outStream);
```

תיאור הפעולה: פקודה זו מדפיסה לערוץ הפלט את כל חברות התחבורה על פי רווחים נטו (הכנסות פחות הוצאות). החברות יודפסו בסדר עולה על פי הרווחים נטו. ההדפסה של כל חברת תחבורה תתבצע באמצעות הפונקציה `prog2_report_company()` המוגדרת בקובץ `prog2_ex3.h`.

פרמטרים:

- tm - מצביע למבנה הנתונים
- outChannel - ערוץ הפלט אליו יש לשלוח את הדווח

ערכי החזרה:

- `TRANSPORT_EMPTY` – אין חברות תחבורה במאגר כלל.

5.6 הגבלות על המימוש

עליכם לממש את המערכת תוך התחשבות במגבלות הבאות:

- מספר חברות התחבורה במערכת **אינו מוגבל**. שימו לב שעליכם לכלול (`#include` "prog2_ex3.h") את הקובץ הזה בקובץ המימוש שלכם על מנת שתוכלו להשתמש בהגדרות שבו.
- **את כל ההדפסות לפלט יש לבצע באמצעות הפונקציות המתאימות המוגדרות ב** `prog2_ex3.h`
- עליכם להקצות את גודל הזיכרון המתאים עבור **כל מחרוזת** שנשמרת בזיכרון. **אין להקצות זיכרון מיותר**
- בכל המקרים, ביציאה מהתוכנית, יש לשחרר באופן מפורש את כל המשאבים שהוקצו למערכת.

5.7 הידור, קישור ובדיקה

5.7.1 פקודת ההידור

על המימוש שלכם לעבור הידור בעזרת הפקודה הבאה:

```
gcc -o transport_manager -Werror -pedantic-errors -L. -lprog2 *.c
```

משמעות הפרמטרים:

- `-o transport_manager` זהו שם קובץ ההרצה המהודר
- `-Werror` התייחס לאזהרות כאל שגיאות – הקוד חייב לעבור הידור ללא אזהרות
- `--pedantic-errors` קבצי הקוד אותם נרצה להדר – קבצי הקוד של התרגיל שכתבתם
- `*.c` קישור עם הספרייה `libprog2.a` המכילה את מימושי ה ADTs
- `-L. -lprog2` המסופקים. על ספרייה זו להיות באותה התיקייה עם קבצי הקוד שלכם.

5.7.2 אופן הידור התכנית

עליכם לכתוב Makefile שיבנה את התכנית תוך התחשבות בתלויות בין המודולים שבה.

5.7.3 בדיקת התרגיל

התרגיל ייבדק בשני אופנים.

בדיקה אחת כוללת מעבר על הקוד ובודקת את איכות הקוד (אי קיום שכפולי קוד, קוד מבולגן ולא קריא "ספגטי", שימוש בטכניקות קידוד "רעות" וכו').
הבדיקה השנייה כוללת את הידור התוכנית המוגשת והרצתה במגוון בדיקות אוטומטיות. על התוכנית לעבור הידור, לרוץ את הבדיקות בשלמותן ולעבור השוואה של קבצי הפלט עם קבצי פלט מצופים.

על מנת לבדוק את תוכניתכם, **קיימת תיקייה בשם tests ובה מסופקים שני מבדקים כאלו עם הפלט הסטנדרטי והשגיאות המצופים עבור כל אחד**. תוכלו להשתמש בתכנה `diff` על מנת להשוות את הפלט שלכם עם הפלט המצופה.
הריצו את אותן הבדיקות עבור הקוד שלכם ע"י מיקום התיקייה בתוך תיקיית העבודה שלכם והרצת `make` מתוך תיקיית הבדיקות (שם מחכה לכם קובץ `Makefile` מתאים למבדקים). ייווצרו בדיוק אותם הקבצים עם הפלט שלכם.
שימו לב! התוכנית שלכם תיבדק בדיקות נוספות. הבדיקות המסופקות מכסות רק חלק בסיסי מהמקרים האפשריים.

בדיקת התרגיל בעצמכם מהווה חלק מהתרגיל!

- כתבו בדיקות נוספות בעצמכם בהתאם למפרט התרגיל
- וודאו את נכונות התוכנית שלכם במקרים כלליים ובמקרי קצה
- מומלץ לחשוב על הבדיקות כבר בזמן כתיבת הפתרון
- העדיפו ליצור מספר רב של בדיקות על פני בדיקה אחת גדולה שיהיה לכם קשה להתמצא היכן הטעויות
- וודאו נכונות הקוד לפני ההגשה גם לאחר שינויים קטנים ולכאורה "לא מזיקים"
- שימו לב לדליפות זיכרון! הם ייבדקו באופן אוטומטי על ידי תוכנה מתאימה (valgrind)

6 הגשת התרגיל

עליכם להגיש את פתרון חלק א בקובץ נפרד (word, pdf) בחלק ב עליכם להגיש אך ורק את הקובץ/קבצים שכתבתם. בפרט אין להגיש את הקבצים המסופקים את פונקציית main שלכם

7 דגשים ורמזים

- מומלץ, עוד לפני תחילת כתיבת הקוד, לצייר סכמתית את מבני הנתונים ואת תיאור המערכת כפי שראיתם בהרצאות ובתרגולים.
- יש להקפיד על ניהול זיכרון נכון. אין להקצות זיכרון מיותר (למשל, זיכרון באורך `MAX_LEN` עבור כל מחרוזת). כמו כן, יש להקפיד על שחרור של זיכרון כאשר אין בו צורך יותר
- יש להקפיד על תכנון נכון של התוכנית וכתיבה נכונה בשפת C. בשלב זה של לימודיכם הנכם מסוגלים ונדרשים לתכנן ולכתוב תכנית בסדר גודל הנתון בעצמכם. הקפידו לבצע חלוקה נכונה למודולים ולפונקציות בהתאם למבנה הלוגי של התוכנית ולא לכתוב פונקציית main אחת גדולה.
- נצפה לראות הפרדה בין פונקציות המממשות לוגיקה מסוימת לאחרות וכן מודולים TransportCompany ו-TransportManager **לכל הפחות**. הנכם יכולים לכתוב מודולים נוספים כראות עינכם.
- כדי למנוע בעיות עם הבודק האוטומטי, ערך החזרה של התוכנית (ביציאה מפונקציית main) צריך להיות תמיד 0.
- יש להקפיד לא להוציא פלט אלא על ידי הפונקציות שסופקו לכם. אין לקרוא קלט מהשתמש!
- במקרה של הסתבכות עם באג קשה:
 - בודדו את הבאג ושחזרו אותו באמצעות מספר מינימלי ככל הניתן של פעולות. ניתן לעשות זאת על ידי מחיקת חלק מהשורות בקוד ובדיקה אם הוא עדיין מתרחש.
 - ניתן להשתמש בהדפסות בריצת התוכנית שמציגות את מצב הריצה (ערכי משתנים, מצביעים וכו').
- חשבו היטב באיזה טיפוס נתונים ראוי להשתמש על מנת לממש כל מודול. בחירה גרועה של טיפוסים נתונים. למשל שימוש ב - Set במקום שבו היה ראוי להשתמש ב - LinkedList (או להיפך) תקשה עליכם את המימוש.
- בתרגיל בית זה אין להשתמש במערכים ואין לממש רשימה מקושרת בעצמכם. הנכם נדרשים להשתמש במודולים המסופקים לכם.